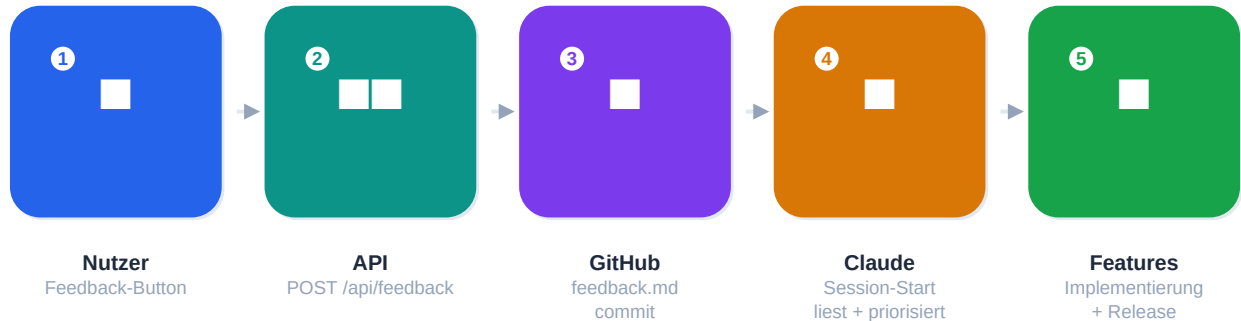


AI-gestützter Feedback-Prozess

Wie Nutzer-Feedback direkt in Features wird – ohne Ticket-System, ohne Meetings.

Der Prozess auf einen Blick



Von der Eingabe bis zum Feature dauert es typischerweise eine Session (< 2 h).

Schritt für Schritt

■ Schritt 1 – Nutzer sendet Feedback

Der eingeloggte Nutzer klickt den **Feedback-Button** (fixed, unten links in der App) und wählt eine Kategorie: *Fehler* · *Feature-Wunsch* · *Allgemein* · *Sonstiges*. Die Eingabe ist auf 2.000 Zeichen begrenzt.

■ Schritt 2 – API speichert & versioniert

Der POST-Request landet bei **/api/feedback**. Die API speichert den Eintrag parallel in zwei Wegen: (a) in die Supabase-Datenbank (Tabelle *feedback*, mit *workspace_id* + *user_id*) und (b) als **Git-Commit** direkt in die Datei *feedback.md* im Repository – via GitHub Contents API. Jeder Eintrag bekommt eine fortlaufende ID (F-001, B-002 ...).

■ Schritt 3 – feedback.md im Repository

Die Datei ist eine strukturierte Markdown-Datei mit einem Eintrag pro Feedback. Format: **ID | Timestamp | Kategorie**, Workspace, Nutzer-E-Mail, Nachricht. Gelöste Einträge bekommen den Marker *Status: bearbeitet*. So ist der komplette Feedback-Verlauf versioniert und nachvollziehbar.

■ Schritt 4 – Claude liest & priorisiert

Beim nächsten Session-Start liest Claude die *feedback.md* automatisch aus (über den Kontext-Inject oder manuell per "Gibt es neues Feedback?"). Claude priorisiert nach Häufigkeit, Workspace-Relevanz und technischer Machbarkeit – und schlägt vor, welche Punkte er als nächstes umsetzt.

■ Schritt 5 – Implementierung & Release

Claude implementiert das Feature direkt im Repo, erstellt einen Commit und pusht auf den Development-Branch. Der GitHub Actions Workflow deployed automatisch auf den Produktiv-Server. Abschließend wird der Feedback-Eintrag auf *Status: bearbeitet* gesetzt und ein Update-Eintrag in **DashboardUpdates.tsx** veröffentlicht – sichtbar für alle Nutzer im Dashboard.

Was du brauchst

Komponente	Beschreibung
GitHub-Repo	Ein Repository, in dem Claude Commits machen darf (Schreib-Zugriff).
GITHUB_FEEDBACK_TOKEN	Personal Access Token mit Scope <i>repo</i> (read + write contents). Als Env-Variable hinterlegen: GITHUB_FEEDBACK_TOKEN .
GITHUB_FEEDBACK_BRANCH	Der Branch, auf den Feedback-Commits landen (z.B. <i>main</i> oder <i>feedback</i>). Default: <i>main</i> .
feedback.md	Die Zieldatei im Repo-Root. Kann leer anlegen – die API erstellt den Header automatisch beim ersten Commit.
Claude Code Zugang	Claude muss das Repo lesen können. Am einfachsten: Claude Code CLI / Claude Code Web mit Repo-Zugriff.

Muster-Prompt: Session-Start mit Feedback-Check

Diesen Prompt kannst du in dein **CLAUDE.md** eintragen – Claude liest ihn bei jedem Session-Start und weiß automatisch, was zu tun ist:

```
## Feedback-Prozess

Beim Session-Start immer zuerst feedback.md lesen und prüfen, ob es neue
Einträge gibt (kein "Status: bearbeitet"-Marker).

Neue Einträge priorisieren:
- ■ Fehler (B-xxx) → sofort ansprechen
- ■ Feature-Wunsch (F-xxx) → nach Häufigkeit / Aufwand abwägen
- ■ Allgemein (G-xxx) → ggf. nur bestätigen

Nach der Implementierung:
1. Feedback-Eintrag in feedback.md mit "Status: bearbeitet" markieren
   und eine **Lösung:** Zeile ergänzen (1 Satz, was umgesetzt wurde)
2. Neuen Eintrag in DashboardUpdates.tsx (UPDATES-Array) ergänzen
   mit ID, Datum, kurzem Titel und nutzerorientierter Beschreibung
3. Commit + Push auf den Development-Branch

Format neuer Feedback-Einträge in feedback.md:
## F-XXX | YYYY-MM-DD HH:MM:SS | ■ Feature-Wunsch
**Workspace:** MeinProjekt
**Nutzer:** user@example.com

[Nachricht des Nutzers]

---
```

Tipps aus der Praxis

Kleiner Button, große Wirkung. Der Feedback-Button ist fixed unten links platziert – immer erreichbar, nie im Weg. Nutzer senden Feedback genau dann, wenn etwas auffällt.

Git als Single Source of Truth. feedback.md liegt im Repo – Claude sieht es ohne extra Tooling. Kein Jira, kein Trello, kein Webhook nötig.

Denormalisierter Kontext. Workspace-Name und E-Mail direkt im Eintrag speichern (nicht nur IDs). Claude muss keine Joins machen.

Resilient by design. API speichert immer in beide Wege (DB + GitHub). Wenn einer fehlschlägt, landet das Feedback trotzdem irgendwo.

Transparenz für Nutzer. Dashboard-Updates-Sektion zeigt pro Eintrag Titel, Datum und Beschreibung – Nutzer sehen, dass ihr Feedback ankommt.